

Multi Paradigm Programming

Michelangelo Prisciano

February 16, 2026

Contents

1	Week 1	1
1.1	Intro	1
1.2	Data Types in C	1
1.3	Strings in C	1
1.4	printf	2
1.5	How to read input from the user	2
1.5.1	scanf	2
1.5.2	gets	2
1.5.3	fgets	2
1.5.4	getline()	2
1.6	Strings manipulation	2
1.6.1	strncpy	2
1.6.2	strncmp	2
1.6.3	strlen	3
1.6.4	strdup	3
1.6.5	atoi and atof	3
1.7	Control Structures and Functions	3
1.7.1	Alternative #1: two-stage declaration	3
1.7.2	Alternative #2: separate header file	3
1.8	Arrays	4
1.9	#define statements	4
2	Week 2	5
2.1	Structs in C	5
2.2	Typedefs	5
2.3	Header files in C	6
2.4	Pointers	7
2.4.1	What is a pointer?	7
2.4.2	Why Pointer Types Matter	8
2.4.3	Call By Value VS Call By Reference	8
2.4.4	Pointers to structs	8
2.5	Dynamic Memory Allocation	9
2.5.1	malloc()	9
2.5.2	calloc()	10
2.5.3	free()	10
2.5.4	sizeof()	10
2.5.5	When malloc() fails	10
2.6	Questions	11

3	Week 3	12
3.1	Stack	12
3.1.1	Initializing the stack	12
3.1.2	Pushing data onto the stack	12
3.1.3	Pop data from the stack	12
3.1.4	Free Up The Stack	13
3.1.5	Peek The Stack	13
3.2	Double Ended Queue	13
4	Week 4	13
4.1	Graphs	13
4.2	Graphs Algorithms	14
4.2.1	Breadth First Traversal	14
4.2.2	Depth First Traversal	14
4.2.3	Dijkstra Algorithm	14
5	Week 5	15
5.1	Time Complexity of Algorithms	15
5.2	File Handling in C	15
5.2.1	Writing to a File	16
5.2.2	Reading from a File	16
5.2.3	Reading Numbers as Text	16
5.2.4	Parsing Lines of Text	16
5.3	From C to C++	17
6	Week 7	18
6.1	Namespaces	18
6.2	Arrays	18
6.3	Functions	19
7	Week 8	20
7.1	Overloading	20
7.2	Object Oriented Programming Concepts	20
7.2.1	Encapsulation	20
7.2.2	Inheritance	20
7.2.3	Polymorphism	21
7.2.4	Abstraction	21
7.3	Operator Overloading	21
8	Week 9	22
8.1	Hash Tables	22
8.2	Chained Hash Tables	22
8.3	Open Addressing	23
8.4	Double Hashing	23
8.5	How to deal with deletion in probing sequences?	24
8.6	Hash Table Size	24
8.7	Rehashing	24
9	Week 10	25
9.1	Templates	25
9.1.1	Function Templates	25
9.1.2	Class Templates	26
9.1.3	Explicit Template Specialization	26
9.2	File I/O	27
9.2.1	Basics	27
9.2.2	Control Modes	27

9.2.3	Examples	28
9.3	Error Handling	29
9.3.1	The Basics	29
9.3.2	Stream Errors	31
10	Libraries Recap	32
10.1	C	32
10.2	C++	33

1 Week 1

1.1 Intro

When writing a program many times we need functions defined in other libraries, for example `stdio.h`: these are imported by including the relevant header files, in this case `<stdio.h>`. Once imported they can be used in the program, this one in particular is one of a range of standard headers provided in C.

Functions have return values. The standard is that "0" is the value returned if the program completed correctly, otherwise another one can be returned.

Projects made up of several `.c` files can be compiled together by executing `gcc file1.c file2.c etc`

The output program name can be personalized by doing `gcc hello.c -o michelangelo-program`

There's a number of flags that can be used with `gcc`:

- `-Wall` → enables all warnings
- `-Werror` → to treat warnings as errors
- `-pedantic` → to ensure compliance with strict C standards
- `-g` → to add debugging information
- `-c` → to compile a file without linking

Compilation errors. Linking errors are due to the compiler not being able to find a function used somewhere in the code, for example.

Runtime errors are logical errors in the program, for example segmentation fault.

C makes use of Snake Case (example, `get_password()`)

1.2 Data Types in C

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes (4 bytes for 32-bit OS)	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615
float	4 byte	1.2E-38 to 3.4E+38 (6 decimal places)
double	8 byte	2.3E-308 to 1.7E+308 (15 decimal places)
long double	10 byte	3.4E-4932 to 1.1E+4932 (19 decimal places)

1.3 Strings in C

There's no string data type in C. We make use of characters arrays terminated by null pointers.

When using something like `printf("Hello my name is %s", myName)`, C will look at the variable `myName` and it will copy into the string all characters up until the null terminator ("0"). If the null terminator isn't there it will keep going. When declaring string literals the null terminator is automatically added.

```
char garbage[] = {'X', 'X', 'X', 'X', 'X'}; // Fill nearby memory
char myName[] = {'M', 'i', 'c'};           // No null terminator
printf("My name is: %s\n", myName);
```

This will produce the following output because printf keeps reading into memory.

```
My name is: MicXXXXX
```

To use booleans we need to import a library.

1.4 printf

Use the printf statement with the right format specifier. For example, printf("The number %d is an integer", %int_variable)

When printf parses the information and it comes at the placeholder, it copies the value of the input parameter int_variable into the string at the given location.

Data type	Format specifier
int	%d
float	%f
double	%lf
char	%c
string	%s

1.5 How to read input from the user

1.5.1 scanf

scanf is used to read information from the user. It stops when it encounters a white space.

scanf is dangerous because it does not check the size of the data coming in, therefore it's prone to memory overflows. A workaround is something like this

```
char name[100];
printf("Please enter your name: ");
scanf("%99s", name);
```

This tells scanf to read only the first 99 characters. scanf returns an integer type, this is the number of items successfully read.

1.5.2 gets

This reads an entire line of text.

1.5.3 fgets

Same but it lets you specify how many characters to be read in.

1.5.4 getline()

1.6 Strings manipulation

1.6.1 strncpy

Copy up to n characters from one string to another one

1.6.2 strncmp

Compare up to n characters between two strings

1.6.3 strlen

Get the length of a string

1.6.4 strdup

Duplicate a string by making another copy in memory

1.6.5 atoi and atof

Convert a string to an int or to a float

1.7 Control Structures and Functions

”return” can be used in a void function to terminate it early
Functions need to be declared in advance in C.

1.7.1 Alternative #1: two-stage declaration

```
#include <stdio.h>

// advance declaration of function
void displayMessage();

int main()
{
    // call a function
    displayMessage();
}

// definition / implementation as before
void displayMessage()
{
    printf("Hello!\n");
}
```

1.7.2 Alternative #2: separate header file

```
#include <stdio.h>
#include "myfunctions.h" // This contains all functions declarations

int main()
{
    // call a function
    displayMessage();
}

// definition / implementation as before
void displayMessage();
{
    printf("Hello!\n");
}
```

The reason why we need declarations in advance is because C uses a **single-pass compiler** (or close to it). When the C compiler reads your code from top to bottom, it needs to know about every function before it's used. If it encounters a function call like `displayMessage()` but hasn't seen its declaration yet, it doesn't know what the return type is, what parameters it takes or whether it even exists.

Java on the other hand uses a **multi-pass compiler**. The Java compiler reads through your entire source code multiple times: the first time it collects all class and method signatures and on the following iterations it resolves references and generates bytecode. This means Java already knows about all your methods before it starts checking method calls, so you can call methods that are defined later in the file or even in other classes.

1.8 Arrays

Arrays are declared like this

```
int numbers[5] = {1, 2, 3, 4, 5}
int numbers[5] = { 0 } // this initialises all elements to 0
int numbers[] = {1, 2, 3, 4, 5} // automatically gets size of array
```

While in Java arrays are objects, in C these are just consecutive memory locations. C also doesn't have a way to get the array size.

When declaring a two-dimensional array we need to put the second dimension in the declaration itself and when passing it around in functions as well.

1.9 #define statements

A constant is defined in the top part of the source file (under `#include` statements)

```
#define ARRAY_SIZE 24 //constant name and constant value
```

Another use is for conditional clauses

```
#define WINDOWS // this defines WINDOWS as in, it exists
[...]
#ifdef WINDOWS
    // windows code specific
#else
    // other code
#endif
```

Some other advanced uses of define statements

```
#define ERROR_MSG "Sorry, an error has occurred"
#define COMMON_CODE for (int i=0; i<10; i++)

int main()
{
    // report an error
    printf("%s\n", ERROR_MSG);

    // perform some common code
    COMMON_CODE
    {
        printf("Value: %d", array[i]);
    }
}
```

2 Week 2

2.1 Structs in C

Despite not being an object-oriented programming language, C supports data structures. For example:

```
struct person {
    char name[30];
    double age;
    bool isVegan;
}

struct person p1;

p1.age = 25.4;
p1.isVegan = true;

printf("The person's age is %f", p1.age);
```

Functions cannot be added to a struct in the same way you could do with an object in Java. However I can define functions to extract information from structs:

```
double getAge(struct person p) {
    return p.age;
}
```

We can even have arrays of structs like:

```
struct person {
    double age;
};

struct person person_array[10];
person_array[0].age = 25.5;
```

2.2 Typedefs

Typedefs (type definitions) are used to give aliases to existing data types in C. For example:

```
typedef int WholeNumber;
[...]
WholeNumber num = 12;
```

These can be used with structs as well in order to enhance code readability:

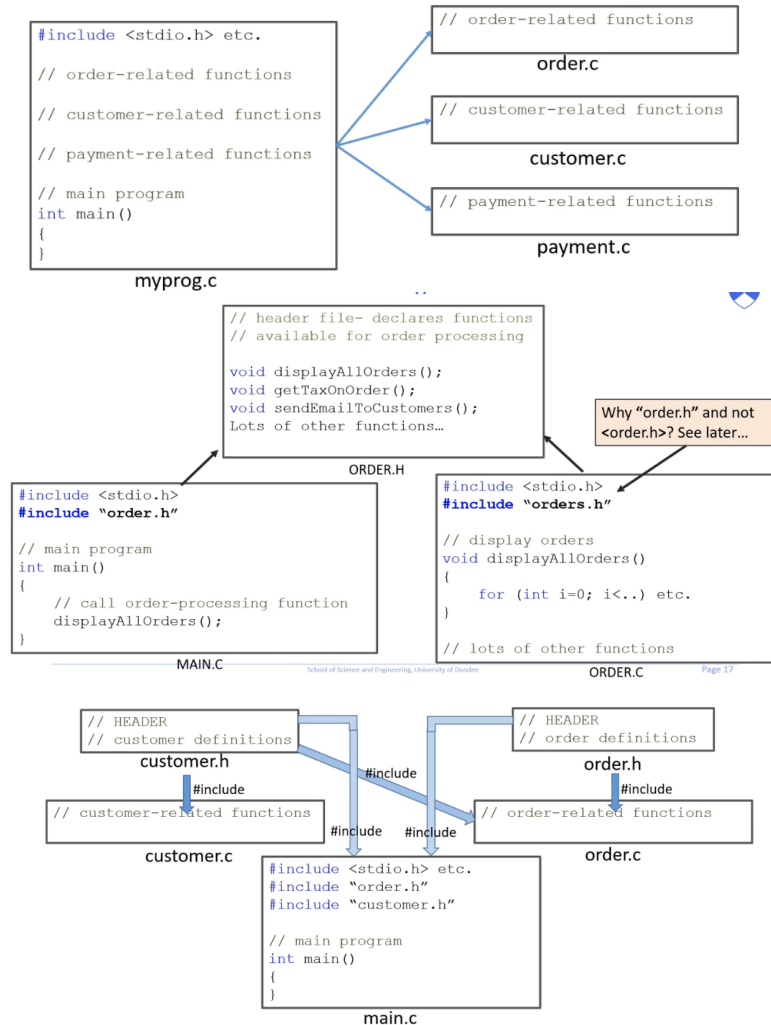
```
typedef struct person Person;
Person p1;
p1.age = 25;
```

Rather than declaring struct and typedef in two separate statements, the two can be combined:

```
typedef struct person {
    double age;
} Person;
```

2.3 Header files in C

Having header files allows us to arrange the program into multiple files to support modular programming.



Standard things to include in header files are definitions, structs/typedefs and functions declarations.

In order to avoid circular dependencies/definitions, we can use a "guard" statement. The portion between `#ifndef` and `#endif` is executed only if `__ORDERS_H__` is not defined.

```

#ifndef __ORDERS_H__
#define __ORDERS_H__

#define MAX_ORDERS 100

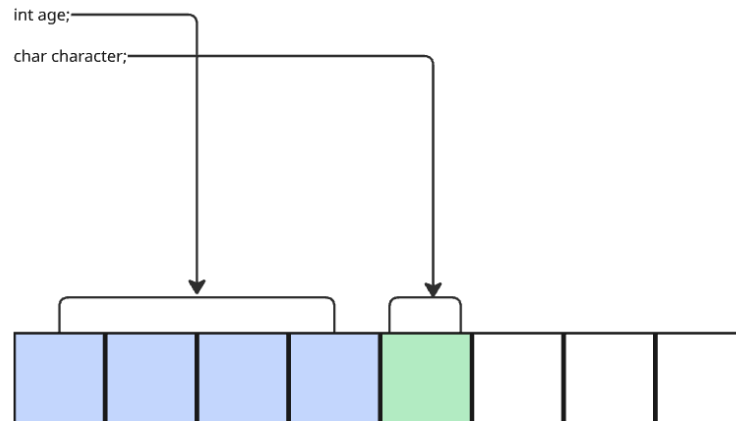
typedef struct order {
    int orderID;
} Order;

void displayOrders(Order orders[]);
void calculateTax(Order orders[]);
#endif
  
```

2.4 Pointers

2.4.1 What is a pointer?

A computer's memory is made up of a sequence of "boxes", each one holding one byte. In this example there are 2 blocks of memory reserved: 4 bytes for the variable age and 1 byte for the variable character.



In the case of arrays, these are a sequence of values that are placed at consecutive locations in memory and accessed through an "offset", the index. Similarly, when declaring a struct the operating system will set aside enough space for the content of that data structure.

Going back to computer memory, each byte has an address to distinguish it from the other bytes in memory and each variable in the program occupies one or more bytes of memory. The address of the first byte is said to be the address of the variable: these addresses can be stored in pointer variables. Put simply, a pointer is just a variable that can hold an address.

```
int *p;
```

This declaration means that `p` is a pointer variable capable of pointing to objects of type `int`. This is important as C requires that every pointer variable only points to objects of the references type.

We can use `&` to find the address of a variable and `*` to gain access to the object that the pointer points to. This statement makes the pointer `p` point to `i`;

```
int i, *p;  
...  
p = &i;
```

Throwback to when we were using `scanf`:

```
int num;  
scanf("%d", &num);
```

Here we were giving `scanf` the address of the variable `num` so that it knows exactly where to go in memory to copy the value input by the user.

Some best practices with pointers include initialising a pointer value to `NULL` when first declaring it and also checking if it points to `NULL` when using it.

2.4.2 Why Pointer Types Matter

Memory addresses themselves are all the same size, typically 4 bytes on 32-bit systems, 8 bytes on 64-bit systems. A pointer variable storing an address to an `int` takes up the same amount of space as a pointer storing an address to a `char` or a `double`.

Therefore why do we need typed pointers? It's all about **pointer arithmetic** and **dereferencing**.

When you do `ptr + 1`, the compiler needs to know how many bytes to advance:

```
int *p = some_address;    // say address 1000
p + 1;                    // moves to 1004 (adds 4 bytes)

char *c = some_address;   // say address 1000
c + 1;                     // moves to 1001 (adds 1 byte)

double *d = some_address; // say address 1000
d + 1;                     // moves to 1008 (adds 8 bytes)
```

Furthermore, when you dereference a pointer with `*ptr`, the compiler needs to know **how many bytes to read**:

```
int *p = some_address;
int x = *p; // read 4 bytes from that address

char *c = some_address;
char y = *c; // read 1 byte from that address
```

We can also define a generic pointer that can point to anything, but we **cannot dereference it or do arithmetic on it** without casting it first, because the compiler doesn't know the size of what it points to.

So the type isn't about the size of the pointer itself, it's about how the compiler interprets the memory being pointed to.

2.4.3 Call By Value VS Call By Reference

Pointers are particularly useful when we need to change the value of a variable in a separate function. Rather than calling the function providing the **value** (call by **value**) we would call the function providing a **pointer** to the value stored in memory (call by **reference**).

2.4.4 Pointers to structs

```
Person p1;
p1.age = 45;

Person* pPerson;
pPerson = &p1;

p1.age = 57;
(*pPerson).age = 88;
```

The last line is first dereferencing the `pPerson` struct pointer and then accessing its `age` field. The parentheses are necessary because the `"."` operator has a higher precedence over `"*"`. `C` provides a shortcut to this operation:

```
pPerson->age = 88;
```

Passing a struct to a function is very inefficient for large structs because an entire copy of the struct will be made. Instead it's preferable using a call by reference and pass a pointer to the function.

When passing a struct to a function we can use the "const" keyword to make the struct read-only.

```
void display_person(const Person *p) {}
```

2.5 Dynamic Memory Allocation

Memory can be allocated in two ways:

- Static → allocated at runtime for names variables, function parameters, etc. This is usually allocated on the **stack**
- Dynamic → allocated "on the fly" during the running of the program. We don't know in advance how much memory we need and this is allocated on the **heap**

High Memory Addresses
Stack
↓ (grows downward)
(free space)
↑ (grows upward)
Heap
Static/Global Variables
Code/Text Section
Low Memory Addresses

2.5.1 malloc()

```
void *malloc(size_t size);
```

malloc() dynamically allocates a specified amount of memory and returns a pointer to that block of memory or NULL if it failed to allocate memory. The pointer returned is of void type: this is because malloc() doesn't know what type of data we are storing in the allocated memory. Therefore we need to cast the return type to what we need.

```
int *numbers;
// Allocate memory for 20 integers
numbers = (int*)malloc(sizeof(int) * 20);
// Access memory
numbers[0] = 12; // Equals to *(numbers + 0) = 12;
numbers[1] = 23;
```

We can even allocate memory for an array of structs

```
Person *p;
p = (Person*)malloc(sizeof(Person) * 10);
p[0].age = 24; // equal to (*p).age = 24
p[1].age = 18; // equal to *(p + 1).age = 18
free(p);
```

Array notation with square brackets automatically de-references variables: the index is merely an offset that is applied to the address.

2.5.2 `calloc()`

Similar to `malloc()` but it initialises all the bytes allocated to zero.

2.5.3 `free()`

This function is used to free the memory allocated by `malloc()` or `calloc()`.

```
void free(void *ptr);
```

2.5.4 `sizeof()`

```
sizeof(int); // This returns "4" (bytes)
sizeof(double); // This returns "8" (bytes)
sizeof(Person); // This returns the number of bytes required for a Person struct
```

This function returns the number of bytes of storage space needed for a variable data type.

2.5.5 When `malloc()` fails

`malloc()` will return `NULL` if it can't allocate memory, therefore we should always check that before proceeding in the code. Extra tips: make pointers `NULL` before using them (to prevent accessing memory by accident) and after freeing them (to prevent accessing free memory by accident).

2.6 Questions

- When doing `int *myInteger = (int*)malloc(sizeof(int))`, isn't `(int*)` redundant?
- I want to understand better the structure of the debugger output with

```
gcc -fsanitize=address -g debug.c
```

3 Week 3

3.1 Stack

3.1.1 Initializing the stack

First we will create some useful structs. Note: when declaring self-referencing structs we need to reference the inner struct with the actual "struct + name".

```
typedef struct Node {
    int score;
    struct Node *next;
} Node;

typedef struct stack {
    Node *top;
} Stack;
```

To create the actual stack we will dynamically allocate some memory and check that malloc() was successful.

```
Stack* create_stack() {
    Stack *pStack = NULL;
    pStack = (Stack*)malloc(sizeof(Stack));
    if (pStack == NULL)
        return NULL;
    pStack->top = NULL;
    return pStack;
}

Node *initialise_node(int score) {
    Node *newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL)
        return NULL;
    newNode->next = NULL;
    newNode->score = score;
    return newNode;
}
```

3.1.2 Pushing data onto the stack

In order to push a new node onto the stack, first we need to point the new node to the current "top" node, then we update the "top" field in the stack variable to point to the new node. This is so that we do not lose any references.

```
void push(Stack *stack, Node *node) {
    node->next = stack->top;
    stack->top = node;
}
```

3.1.3 Pop data from the stack

```
void pop(Stack *stack) {
    Node *poppedNode = stack->top;
    stack->top = stack->top->next;
    free(poppedNode);
}
```

3.1.4 Free Up The Stack

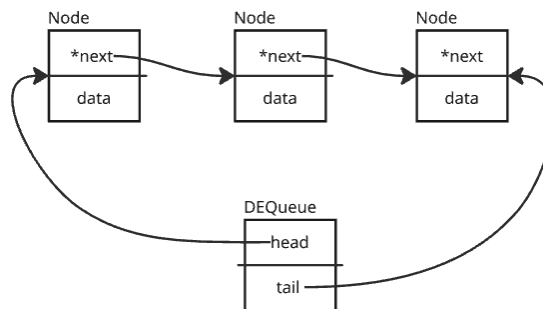
```
void free_stack(Stack *stack) {
    Node *pNodeToFree;
    while (stack->top != NULL) {
        pNodeToFree = stack->top;
        stack->top = pNodeToFree->next;
        free(pNodeToFree);
    }
    free(stack);
}
```

3.1.5 Peek The Stack

```
int peek(Stack *stack) {
    return stack->top->score;
}
```

3.2 Double Ended Queue

This is a data structure similar to a queue, except that elements can also join from the front of the queue rather than just from the back.



4 Week 4

4.1 Graphs

A graph is a data structure that consists of a finite collection of vertices and edges that represent the links or connections between the vertices. They can be directed or undirected, depending on what is the direction of travel between the nodes. Edges can have weights associated with them. Two common ways of representing graphs through data structure are **adjacency matrices** and **adjacency lists**.

Undirected graphs record edges in both directions. When we have a weighted graphs, these are recorded in the adjacency matrix (and all other entries would be 0 or -1, depending on the situation).

How to implement an adjacency matrix in C?

```
#define VERTEX_NUM 10
int adjacencyMatrix[VERTEX_NUM][VERTEX_NUM]
```

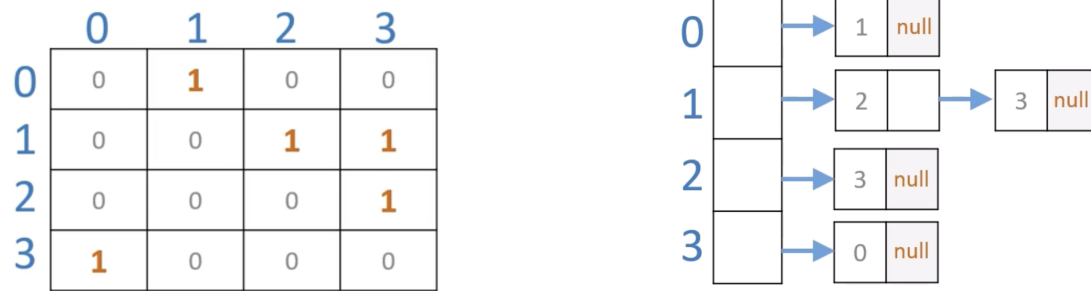


Figure 1: Adjacency Matrix VS Adjacency List

4.2 Graphs Algorithms

4.2.1 Breadth First Traversal

This algorithm explores the breadth of immediate connections from a given node.

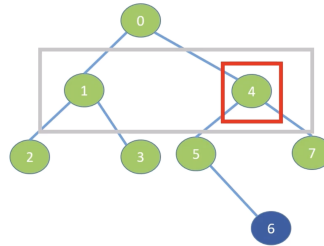


Figure 2: Breadth First Search

4.2.2 Depth First Traversal

This algorithm, on the other hand, follows a single path for as much as it can, then it backtracks and follows an alternative path. To implement this we can use an adjacency matrix, an array called `visited` and a second array which we'll use as a stack.

4.2.3 Dijkstra Algorithm

This algorithm is used to find the shortest path from a given node to any node in the graph. Here's a summary of the steps:

- Record distance in the Dijkstra table for `currentNode` (this will be 0)
- Identify `connectionNode` at most 1 step away (iterate through the adjacency list / matrix the outwards connections)
- Check if the `connectionNode` was visited, if not calculate the cumulative distance to get to `currentNode` plus the distance between `currentNode` and `connectionNode`. If this distance is lower than the one recorded in the Dijkstra table for `connectionNode`, update the distance and the predecessor in the table.
- Once finished with the connected nodes, mark `currentNode` as visited
- The next node we visit will be the unvisited node with the shortest distance recorded in the table
- *all over again*

5 Week 5

5.1 Time Complexity of Algorithms

We express time complexity of an algorithm through the "Big O Notation". For example, if we have a sorted array (from lowest to highest) the time complexity will be $O(1)$. If the array isn't sorted, the complexity will be $O(n)$ (also known as linear time). Here's some complexity for common algorithms:

- Binary search
 - Best case: $O(1)$ (if the value found is the first midpoint in the array)
 - Worst case: $O(\log n)$
- Add node to a (non-empty) tree
 - Best case: $O(1)$ (the tree is empty)
 - Worst case: $O(n)$ (the tree is unbalanced)
 - Average case: $O(\log n)$ (the tree is reasonably balanced)
- Bubble sort
 - Best case: $O(n)$ (list is already sorted)
 - Worst case: $O(n^2)$

5.2 File Handling in C

C uses the concept of streams, which is a flow of information that can go from a file to our program (input stream) or from our program to a file (output stream). The following are built-in streams:

- **stdin**: this is the standard input device, for example the keyboard
- **stdout**: this is the standard output device, for example the screen
- **stderr**: this is the standard output for errors. It's usually mapped to the **stdout** but it can be mapped to something else

We initialise a file stream with the following code:

```
# FILE *fopen(const char *filename, const char *mode);
# int fclose(FILE *filePtr);

#include <stdio.h>
File *fp = fopen("filename.txt", "r");
[...]
fclose(fp);
```

These are the file access modes available for **fopen()**:

File access mode	Description
"r"	Open a file for reading. The file must exist.
"w"	Open a file for writing. The file is created if it doesn't exist. If a file of the same name already exists, it is truncated (made empty).
"a"	Append data at the end of an existing file. The file is created if it doesn't exist
"r+"	Open an existing file for reading and writing.
"w+"	Create a new file for reading and writing.
"a+"	Open a file for reading and appending.

5.2.1 Writing to a File

- `int fputc(int c, FILE *fp)`: write an individual char to a file
- `int fputs(const char *s, FILE *fp)`: write a (null-terminated) character array (string) to a file. Returns non-negative value if successful
- `int fprintf(FILE *fp, const char *format)`: same as "printf" but writes to a file instead of `stdout`. Returns the number of characters written if successful

5.2.2 Reading from a File

- `int fgetc(FILE *fp)`: reads an individual char from a file. Returns the character read if successful or EOF if it failed
- `char *fgets(char *s, int n, FILE *fp)`: reads a line of text from a file up to a maximum of "n" characters and stores content in the string parameter
- `int fscanf(FILE *fp, const char *format)`: same as "scanf" but reads from a file instead of `stdin`

5.2.3 Reading Numbers as Text

```
char line[256];
while (fgets(line, 256, fp) != NULL) {

    // Convert string to integer
    int number = atoi(line);
}
```

5.2.4 Parsing Lines of Text

```
char *strtok(char *str, const char *delimiter)
```

This function breaks the input string into a series of substrings separated by a given delimiter. It returns a pointer to the first substring found or NULL

```
str = strtok(text, delimiter);
while (str != NULL) {
    printf("%s", str);

    // Call strtok again to get the next substring within the line of text
    // Passing NULL will make strtok use the previous reference given
    str = strtok(NULL, delimiter);
}
```

5.3 From C to C++

What are the main differences between C and C++?

- OOP: C++ is object oriented (classes and objects) while C isn't (it uses structs and functions)
- Strings: C++ supports strings as a data type while C can only use character arrays
- Booleans: C++ natively supports booleans, while in C you need to `#include <stdbool.h>`
- Input/Output Streams: C++ uses `cin` and `cout` while C uses `printf` and `scanf`
- Memory Allocation: C++ uses `new` and `delete`, while C uses `malloc` and `free`
 - `(int*)malloc(sizeof(int) * 10)` and then `free(array)`
 - `int *array = new int[10]` and then `delete [] array`
- Pointers: C++ provides "references" as well (call by value VS call by reference)
- File Handling: C++ uses stream classes/objects (like `ofstream` and `ifstream`) while C uses a pointer to a file stream (`FILE *fp`)
- Namespaces: C++ uses namespaces, which are a way of grouping together related objects and classes
- Collection Classes: C++ has a wide range of collection classes, for example `vector<type>` (which is an extensible array)
- Exception Handling: C++ supports try/catch statements
- Templates: C++ supports templates, which are implementations that can work on generic data types
- `sizeof(char)`: `sizeof(char)` is 4 in C and 1 in C++

6 Week 7

6.1 Namespaces

Namespaces are a feature added in C++. These can be custom as well:

```
#include <iostream>
using namespace std;
namespace other {
    int x = 24;
}

int main() {
    int x = 48;
    cout << "other::x = " << other::x << endl;
    cout << "x = " << x << endl;
}
```

Output:

```
other::x = 24
x = 48
```

Using different namespaces with the same declared variable will result in a compile time error.

```
#include <iostream>
using namespace std;
namespace other { int x = 24; }
namespace mine { int x = 48; }
using namespace other;
using namespace mine;

int main() {
    cout << "x = " << x << endl;
}
```

Error:

```
04-conflict.cpp:9:23: error: reference to 'x' is ambiguous
    cout << "x = " << x << endl;
                        ^
```

6.2 Arrays

How do we declare arrays?

```
int nums[6];
int nums[6] = {1, 2, 3, 4, 5, 6};
int twoD[2][3] = { {1, 2, 3}, {4, 5, 6} };
```

Vectors are a C++ implementation that can be used through `#include <vector>` and has the following functions:

- `size()` - Return size
- `resize()` - Change size
- `empty()` - Test whether vector is empty
- `assign()` - Assign new values to the vector elements by replacing old ones

- `front()` - Access first element
- `back()` - Access last element
- `push_back()` - Add element at the end
- `pop_back()` - Delete last element
- `clear()` - Clear content

6.3 Functions

`getline(cin, str)` is a function that can be used to read from an input stream into the string parameter until it encounters a new-line character.

```
string name;  
cout << "Please enter your name: ";  
getline(cin, name);  
cout << "Thanks, " << name << endl;
```

7 Week 8

7.1 Overloading

We can define multiple classes with the same name as long as their parameters differ by number, data type or both.

7.2 Object Oriented Programming Concepts

We will explore the main concepts in object oriented programming.

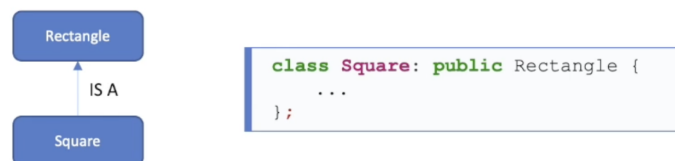
7.2.1 Encapsulation

Encapsulation is a way of hiding details in order to protect them from outside manipulation. We do this by putting a box around something in order to protect its contents: in the case of a class, the class is the wrapper around data and functions.

7.2.2 Inheritance

Inheritance allows us to define a class in terms on another class: in particular, all members of the *derived* class will inherit all members of the *base* class. The access specifier can be either public, protected or private: it's used in order to change access for inherited members when accessed **outside** the derived class.

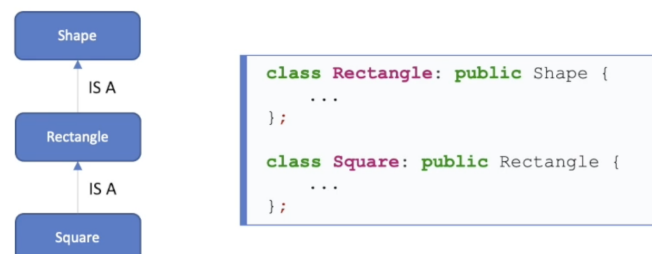
Single inheritance is the classic one.



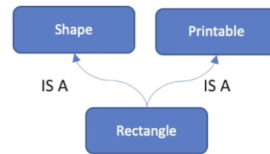
With **hierarchical** inheritance multiple classes are derived from a single parent class.



With **multi-level** inheritance we can have a derived class from a derived class.



With **multiple** inheritance a single derived class can inherit members from multiple base classes.



```

class Rectangle: public Shape, public Printable {
    ... // inherits from both base classes
};
  
```

This table describes how the access changes in derived classes based on the specifier.

Base class	Public	Protected	Private
public	public	protected	private
protected	protected	protected	private
private	inaccessible	inaccessible	inaccessible

7.2.3 Polymorphism

In a programming language that supports polymorphism objects of classes belonging to the same hierarchical tree (inherited from a common base class) *may* have functions with the same name but each one with different behaviors. For example, all Shape classes have a function to calculate the area but it is calculated differently depending on the shape.

7.2.4 Abstraction

Abstraction is a way of hiding details in order to simplify an interface: we don't need to know all the details of how the car works to drive it, we can just use the many abstractions provided (for example, steering wheel, etc). In the same way, if we define an array or a vector whoever uses the class doesn't need to care as long as they have the right functions to retrieve and store data.

7.3 Operator Overloading

C++ allows us to change the functionality of most operators.

```

Rectangle operator+(const Rectangle& r1, const Rectangle &r2) {
    Rectangle rect;
    rect.width = this->width + r.width;
    rect.height = this->height + r.height;
    return rect;
}
  
```

Other operators can all be overloaded. Relational operators are similar but must return a boolean value. The increment and decrement operators need to be overloaded for both postfix and prefix usage.

8 Week 9

8.1 Hash Tables

A **hash table** is an unordered array of fixed size whose elements are accessed through a key. It offers a combination of efficient lookup, insert and delete operations. A **hash function** maps a key to a hash table index. The notation used from now on will be: $h()$ for the hash function, k for the key, T for the hash table and $|T|$ for the size of the hash table (i.e. the number of indexes). When a hash table can guarantee that no two keys will generate the same index the resulting hash table is called **directly addressed**. A **collision** occurs when a hash function maps different keys to the same index. They're unavoidable when the number of potential key values is greater than the size of the table.

Some characteristics of a good hash function are that:

- it minimizes the chance of collisions by distributing keys uniformly over the table indexes
- is fast
- is deterministic (i.e. the same key is always mapped to the same index)

In the case of integer keys a simple hash function is $h(k) = k \bmod |T|$, also called **modular hash function**. It's important to choose the hash function carefully: for example, when using the modular hash function the size of the hash table should be a prime number to guarantee a uniform distribution of keys over the indexes.

In the case of string keys (S) we have some options. We can multiply the ASCII value of the first three characters of each word by a prime number:

$$h(S) = \text{int}(S[0]) + 3 \cdot \text{int}(S[1]) + 5 \cdot \text{int}(S[2]) \bmod |T|$$

or alternatively compute the following sum ($p = 31$ is a popular choice):

$$S[N-1] + S[N-2] \cdot p + S[N-3] \cdot p^2 + \dots + S[0] \cdot p^{N-1} \bmod |T|$$

How to handle collisions?

8.2 Chained Hash Tables

In a **chained hash table** each slot has a linked list and whenever a collision occurs the value is inserted at the head or tail of the list located at $T[h(k)]$.

The time complexity of this approach is determined by the number of items in the linked list that contains the key. In the *worst-case scenario* all entries are inserted into the same linked list which would make the cost of lookup $O(N)$ where N is the number of keys in the table.

In the *average case* scenario the cost of lookup depends on the average number of keys in the linked list. We can define the **load factor** $\lambda = N/M$, where N is the number of keys in the table and M is the number of slots. If the keys are distributed uniformly, then the number of keys in a slot is with high probability close to λ . The cost of lookup will be $O(1 + \lambda)$: 1 to access the slot, λ to search the list until the end. If $N \in O(M)$, then the average cost of lookup is $O(1)$. This is because to maintain constant-time operations, we need to keep the hash table size proportional to the number of keys. It's also why hash tables typically resize (double their size) when the load factor gets too high.

Some of the disadvantages are that linked lists could get long, leading to poor performance. A bad hash function could put all elements into one linked list, but a good one will make a chained hash table perform well even if the number of keys is larger than the number of slots.

8.3 Open Addressing

In open addressed hash tables collisions are handled by moving keys to a different place in the table following a fixed strategy. We call *probe sequence* a sequence of slots in a hash table to be looked into while searching for an empty slot to store an element. **Linear** probing will look at the subsequent slot in the table if the slot is occupied, i.e. $h(k) + i$:

$$h(k) + 0, h(k) + 1, h(k) + 2, \text{ etc}$$

while **quadratic** probing will look at $h(k) + i^2$:

$$h(k) + 0, h(k) + 1, h(k) + 4, h(k) + 9, \text{ etc}$$

However, quadratic probing may fail to find an empty slot if the table is not at least half-empty.

8.4 Double Hashing

With this approach we use a second hash function $g(k)$ (which must never be equal to 0) to determine the probing sequence. For example, a probing sequence would look like:

$$h(k), h(k) + g(k), h(k) + 2*g(k), h(k) + 3*g(k)$$

Telephone directory: $|T|=13$

$h(k)$ = Hash first letter of key as below:

0	1	2	3	4	5	6	7	8	9	10	11	12

a	b	c	d	e	f	g	h	i	j	k	l	m
n	o	p	q	r	s	t	u	v	w	x	y	z

$h'(k)$ = Hash **second** letter of key as below:

1	2	3	4	5	6	7	8	9	10	11	12	13

a	b	c	d	e	f	g	h	i	j	k	l	m
n	o	p	q	r	s	t	u	v	w	x	y	z

Slots for "John" = 9,11,0,...

Slots for "William" = 9,5,1,...

Figure 3: Example with double hashing

One major advantage of double hashing over linear and quadratic probing is that it's very unlikely that the value of $h(k)$ and $g(k)$ is the same for both keys. This minimizes repeated collisions, all assuming that we're using two good hash functions.

8.5 How to deal with deletion in probing sequences?

Deletion can break the search with probing sequences since search stops whenever we end up in an empty slot. Consider the following example:

0	1	2	3	4	5	6	7	8	9
49			13	14	23	16	3	28	29

Assume we carry out the following operations: search(23), delete(14) and search(23). Since the element 14 in slot 4 was deleted, when searching again for 23 we would stop at the empty slot because we don't want to search the whole table.

The simple solution is to use **lazy deletion**: when we delete an element we mark the slot as deleted (D) rather than empty. Then when looking for a key we continue when we hit a slot that's deleted and stop when we hit an empty slot.

Over a long period of time we might end up with an hash table that has a lot of spots marked as deleted, we can deal with this by rehashing.

8.6 Hash Table Size

We already talked about λ , the load factor. Assuming an ideal hash function the average number of lookups to insert a key in an open addressed hash table is $1 / (1 - \lambda)$.

λ	Average # lookups
0.2	1.25
0.5	2
0.8	5
0.9	10

In order to maintain good performance the rule of thumb for open addressed tables is to double the table size when $\lambda > 0.5$ while for chained hash tables the table size should be doubled when $\lambda > 0.9$.

8.7 Rehashing

Rehashing is a process through which we move all keys in a hash table T to another hash table Z. We maintain both tables, look for keys first in Z (and then in T), move keys from T to Z when the system is idle or whenever a key is looked for and found to be in T.

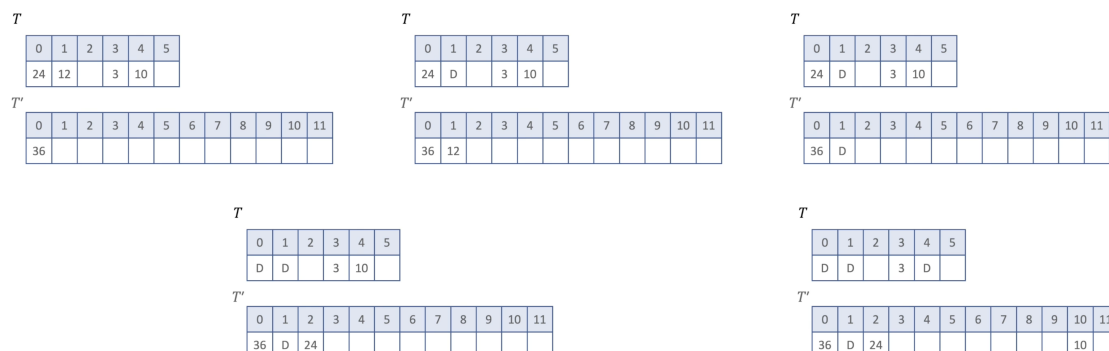


Figure 4: Hash table operations: insert 36, lookup 12, delete 12, lookup 24, and lookup 10

9 Week 10

9.1 Templates

9.1.1 Function Templates

Function templates allow us to use the same function definition with arguments for different types. When a built-in type is used as the parameter type the compiler figures out what type to use. If we're using other types or we want to enforce a certain type, we can specify the type to use. Non-template functions are "first class citizens": a plain old non-template function that matches the parameter types as well as any function template will be selected over an otherwise-just-as-good function template. If there are no first-class citizens to choose from that are at least as good, then function base templates as the second-class citizens get consulted next and the template that gets selected depends on which matches best and is the "most specialized".

```
#include <iostream>
using namespace std;
template <typename T>

T getmin (T a, T b) {
    T result;
    result = (a<b)? a : b;
    return (result);
}

int main() {
    cout << getmin(1, 2) << endl;
    cout << getmin(2.6, 3.7) << endl;

    // Specify type
    cout << getmin<float>(2.6, 3.7) << endl;
    return 0;
}
```

We can also define function templates that accept more than one type parameter, simply by specifying more template parameters between the angle brackets.

```
#include <iostream>
using namespace std;
template <typename T, typename U>
T getmin (T a, U b) {
    return ((a<b) ? a : b);
}

int main() {
    // Specify type
    cout << getmin<float, float>(2.6, 3.7) << endl;
    return 0;
}
```

9.1.2 Class Templates

Template classes encapsulate a similar concept. We can use a generic type inside our classes.

```
template <class T>
class Rectangle {
    T width;
    T height;
public:
    Rectangle(T w, T h){
        width = w;
        height = h;
    }
};
```

9.1.3 Explicit Template Specialization

Explicit template specialization is used when the behavior of a template function is different for particular types from the generic behavior.

```
#include <iostream>
using namespace std;

template <class T>
class mycontainer {
    T element;
public:
    mycontainer (T arg) {element=arg;}
    T increase () {return ++element;}
};

template <>
class mycontainer <char> {
    char element;
public:
    mycontainer (char arg) {element=arg;}
    char uppercase ()
    {
        if ((element>='a')&&(element<='z'))
            element+='A'-'a';
        return element;
    }
};

int main () {
    mycontainer<int> myint (7);
    mycontainer<char> mychar ('j');
    cout << myint.increase() << endl;
    cout << mychar.uppercase() << endl;
    return 0;
}
```

9.2 File I/O

9.2.1 Basics

The standard C++ library for file handling is `fstream` (`#include<fstream>`). A `filestream` object must be created for each file that you wish to open. This will be either an `ofstream` (output `filestream`) for outputting data to a file (used like `cout`)

```
ofstream objectname("file-name");
```

or an `ifstream` (input `filestream`) for inputting data from a file (used like `cin`)

```
ifstream objectname("file-name");
```

Before writing to a `filestream` it's a good habit to test that the `filestream` was actually created with. We should also close the `filestream` once we're done with it.

```
// Check filestream was created
if (!object_name)

// Close filestream after usage
object_name.close()
```

9.2.2 Control Modes

When a `filestream` object is created the parentheses following its name can optionally contain additional arguments, specifying a range of control modes.

`std::ios::out` - opens to write output
`std::ios::in` - opens to read input
`std::ios::app` - opens to append output at the end of an existing file
`std::ios::trunc` - truncate the existing file
`std::ios::ate` - open without truncating, data can be written anywhere
`std::ios::binary` - treat the file as binary format rather than text, so the data may be stored in a non-text format

Multiple modes can be appended if separated by `|` (bitwise or) for example:

```
ofstream objectname("filename", ios::out | ios::binary);
```

9.2.3 Examples

write

```
#include <fstream>
#include <string>
#include <iostream>
using namespace std;

int main() {
    string info = "ABC";

    ofstream writer( "file.txt" );

    if( ! writer ) {
        cerr<<"ErrOpeningFile4Output"<<endl;
        return 1;
    }

    writer << info << endl << 123 << endl;
    writer.close();

    return 0;
}
```

readchar

```
#include <string>
#include <fstream>
#include <iostream>
using namespace std ;

int main() {
    char letter;
    int i=0;

    ifstream reader( "file.txt" );

    if( ! reader ) {
        cout << "ErrOpeningInputFile" << endl;
        return 1;
    }

    while(reader.get(letter)) {
        cout << letter;
        i++;
    }

    reader.close();
    cout << "Iterations: " << i << endl;

    return 0;
}
```

append

```
#include <fstream>
#include <string>
#include <iostream>
using namespace std;

int main() {
    string info = "This is appended to the file.";

    ofstream writer( "file.txt" , ios::app );

    if( ! writer ) {
        cerr<<"ErrOpeningFile4Output"<<endl;
        return 1;
    }

    writer << info << endl;
    writer.close();

    return 0;
}
```

readline

```
#include <string>
#include <fstream>
#include <iostream>
using namespace std ;

int main() {
    string line;
    int i=0;

    ifstream reader( "file.txt" );

    if( ! reader ) {
        cout << "ErrOpeningInputFile" << endl;
        return 1;
    }

    while(getline(reader, line)) {
        cout << line << endl;
        i++;
    }

    reader.close();
    cout << "Iterations: " << i << endl;

    return 0;
}
```

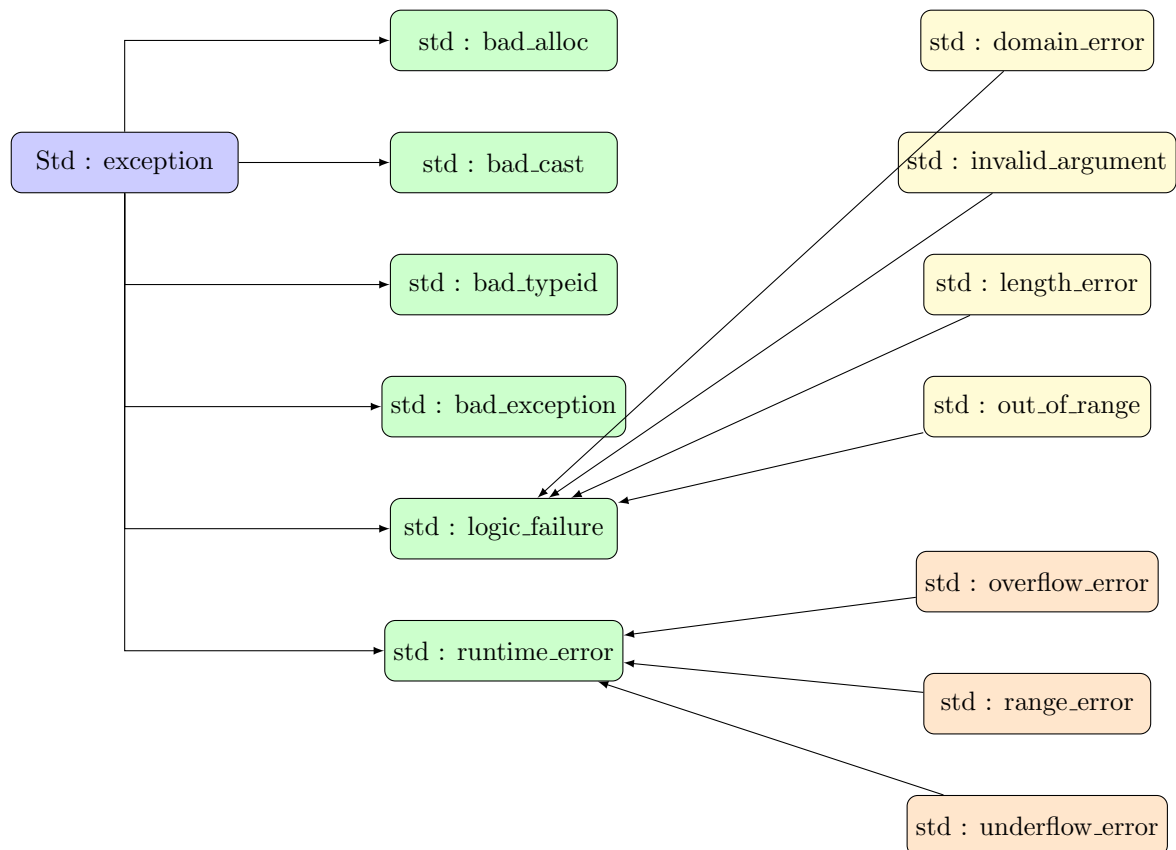
9.3 Error Handling

9.3.1 The Basics

While the compiler spots compile-time errors, it does not detect runtime errors. This can be dealt with by using `try-catch` blocks.

```
#include <iostream>
using namespace std;
int main () {
    try {
        throw 10;
    } catch (int e) {
        cout << "An exception occurred: #" << e << endl;
    }
    return 0;
}
```

Here is a list of standard exceptions defined in `std::exception`.



An example using the `out_of_range` exception.

```
#include <iostream>
#include <exception>
#include <vector>
using namespace std;

int main() {
    vector<int> myvector(10);
    try {
        myvector.at(20)=1;
    }
    catch (const std::out_of_range& oor) {
        cout << "Out of Range error" << endl;
    }
    return 0;
}
```

We can also have multiple catch blocks.

```
try {
    // code which may throw an exception
} catch (const std::exception& e) { //standard exception
    // do something
} catch (type object) { // your own exception
    // do something
} catch (...) { // catches any exception
    // if this is present, it must be the
    // last catch block
}
```

9.3.2 Stream Errors

`std::cerr` is an object of class `ostream` that represents the standard error stream. Stream objects, e.g., `cin`, `cout`, have error state flags. The `good()`, `bad()`, `fail()`, and `eof()` methods return the value of those flags (of type `bool`).

- `bad()`—read/write error
 - Example: `cin.bad()`, `cout.bad()`
 - Indicates loss of integrity of the underlying input/output sequence
 - E.g., device failure, memory shortage, the stream buffer object throws an exception
 - This flag typically indicates that an unrecoverable error has occurred.
- `fail()`—logical/semantic error
 - Example: `cin.fail()`, `cout.fail()`
 - Failed to read or write the expected character
 - Occurs when trying to read into an incompatible type, but also reading past the end of a stream (`eof`)
 - For files failing to open a file sets the fail error flag
 - This type of error is usually recoverable
- `eof()` – end of file flag is set
 - Example: `cin.eof()`
 - The input stream has reached the end.
 - This is not necessarily an error, just an indication that there is no more data to read.
- `good()` – everything's ok!
 - Example: `cin.good()`, `cout.good()`
 - Not an actual error state, equal to `!(bad() || fail() || eof())`

```
int main() {
    int a;
    cout << "Enter an integer: ";
    cin >> a;
    cout << "Your input was: " << a << endl;
    cout << "Good Fail Bad EOF bits:" << endl;
    cout << cin.good() << cin.fail() << cin.bad() << cin.eof() << endl;
    if (!cin) {
        cerr << "Input error." << endl;
        return 1;
    }
    return 0;
}
```

```
Enter an integer: 2
Your input was: 2
Good Fail Bad EOF bits:
1000
```

```
Enter an integer: r
Your input was: 0
Good Fail Bad EOF bits:
0100
Input error.
```

10 Libraries Recap

10.1 C

10.2 C++